

Experimental Evaluation of CPU, Memory and Disk Bottlenecks and Tuning Strategies in Linux

Kartikeya Dimri
Integrated M.tech CSE
IIIT Bangalore
Bengaluru, India
kartikeya.dimri@iiitb.ac.in

Ayush Mishra
Integrated M.tech CSE
IIIT Bangalore
Bengaluru, India
ayush.mishra129@iiitb.ac.in

Harsh Sinha
Integrated M.Tech CSE
IIIT Bangalore
Bengaluru, India
harsh.sinha@iiitb.ac.in

Prof. B. Thangaraju
CSE
IIIT Bangalore
Bengaluru, India
b.thangaraju@iiitb.ac.in

Abstract—This paper presents an experimental study of Linux performance tuning across CPU, memory, and disk I/O subsystems under controlled stress workloads. The study evaluates how subsystem-specific tuning parameters affect system behavior under CPU contention, memory oversubscription, and I/O-intensive workloads. The results show that targeted tuning improves execution efficiency, reduces system pressure, and lowers latency compared to adverse baseline configurations. The experiments further demonstrate that Linux performance bottlenecks are highly workload- and subsystem-dependent, making workload-aware tuning more effective than generic configuration changes. Overall, the work provides empirical insights into practical and reproducible Linux performance tuning strategies.

Index Terms—Linux performance evaluation and tuning, CPU scheduling, memory management, disk I/O, system optimization

I. INTRODUCTION

Linux is a powerful and flexible operating system, widely used in servers, embedded systems, and even personal computers [1]. Its flexibility and extensive configurability enable it to support diverse workloads; however, this general-purpose design often leads to suboptimal performance when default configurations are applied to workload-specific scenarios. Performance degradation typically arises from inefficiencies across key subsystems of an OS such as CPU, memory, and disk I/O handling and network, each of which plays a critical role in overall system behavior. [2]

Prior research has extensively explored Linux performance tuning through different approaches. A substantial part of this prior work has focused on developing measurement and monitoring tools for real-time observation of critical system resources, including CPU, memory, network, and I/O behavior [3], [4]. Traditional methods rely on heuristic or expert-driven tuning, where system administrators manually adjust kernel parameters based on experience. While effective in specific contexts, these approaches are time-consuming and difficult to generalize. More recent work has focused on automated tuning techniques using machine learning and optimization, where systems classify workloads, dynamically adjust parameters, or explore large configuration spaces to improve performance. For example, KernTune applies statistical learning methods to

self-tune Linux kernel performance [5], while KconfigTune searches the Linux kernel configuration space for higher-performing settings [6]. Recent agentic approaches such as OS-R1 further extend this direction by using reinforcement learning for kernel tuning [7]. Similarly, research in storage systems has investigated the impact of I/O schedulers and parameter configurations, with machine-learning-based methods used to recommend better settings for high-concurrency workloads [8]. Despite these advances, such approaches often introduce additional complexity, require significant training data, or emphasize automation over clear experimental understanding of individual tuning decisions.

A key limitation in existing work is the lack of controlled experimental analysis across individual subsystems. Many studies evaluate performance improvements within integrated or automated frameworks, making it difficult to attribute gains to specific tuning strategies. In addition, the reliance on automation can obscure the underlying relationship between configuration parameters and system performance. As a result, there is a need for simpler, experimentally grounded studies that directly demonstrate the impact of tuning decisions without depending on complex automation mechanisms.

Motivated by these challenges, this work focuses on a systematic experimental evaluation of performance bottlenecks and tuning strategies in Linux. Rather than proposing an automated tuning system, the goal is to demonstrate that carefully selected tuning parameters can produce measurable performance improvements under controlled conditions. The study examines CPU, memory, and disk subsystems independently, allowing clearer observation of their individual behavior and bottlenecks. Furthermore, experiments are conducted in separate environments appropriate to each subsystem, ensuring that results reflect realistic constraints rather than a single unified setup.

In this paper, we evaluate the effectiveness of tuning strategies across different workload types, including CPU-bound, memory-intensive, and I/O-intensive scenarios. For CPU performance, the impact of affinity and scheduling policies under contention is analyzed. For memory performance, kernel parameters such as swappiness, dirty write-back thresholds, and Transparent Huge Pages [9] are evaluated under varying pressure conditions. For disk I/O, the role of I/O schedulers

and read-ahead configurations is examined across sequential, random, and mixed workloads.

The main contributions of this work are as follows:

- A systematic experimental evaluation of Linux CPU, memory, and disk subsystem behavior under controlled stress workloads.
- An analysis of the impact of selected tuning parameters on execution time, memory pressure, and disk I/O performance.
- Experimental evidence that workload-aware tuning produces measurable improvements over adverse baseline configurations.
- A reproducible experimental framework for studying subsystem-specific Linux performance tuning.

II. METHODOLOGY AND EXPERIMENTATION

This section describes the methodology used to evaluate Linux performance tuning across three subsystem targets: CPU scheduling, memory management, and disk I/O. Each subsystem is studied independently under controlled stress workloads so that bottlenecks can be observed directly and the impact of tuning strategies can be assessed without conflating effects across domains. The CPU experiments are conducted on a physical machine to avoid virtualization effects on scheduling, affinity, and context-switch behavior, whereas the memory and disk experiments are conducted in a virtual machine to provide controlled and reproducible conditions for swap pressure, cache behavior, and I/O configuration studies.

TABLE I
SYSTEM SETUP USED IN THE EXPERIMENTS

Item	Configuration
CPU experiment	Physical machine; Intel Core Ultra 5 125H; 14 cores, 18 CPUs; up to 4.5 GHz
Memory and disk experiments	VMware virtual machine; Intel Xeon Silver 4116 @ 2.10 GHz; 8 vCPUs
Operating system	Ubuntu-based Linux

We followed a common experimental pipeline across all subsystems, as follows. First, a deliberately suboptimal baseline configuration was established to expose worst-case or degraded behavior for the target subsystem. Next, workload-specific stress tests were executed to trigger CPU contention, memory pressure, or I/O intensity, depending on the experiment. During execution, relevant system metrics were monitored to evaluate observed performance characteristics and bottleneck behavior. Tuning configurations were then applied manually rather than through automated search or adaptive control. Finally, the effect of tuning was assessed by comparing system behavior before and after the configuration changes under comparable workload conditions.

A central insight guiding this study is that performance bottlenecks in Linux are subsystem-specific. Improvements in CPU scheduling do not eliminate memory-management bottlenecks, and disk I/O tuning does not resolve contention in the

processor or memory hierarchy. Effective optimization therefore requires targeted, workload-aware tuning decisions for each subsystem rather than generalized settings or automation-centric approaches. The following subsections provide the subsystem-specific experimental descriptions.

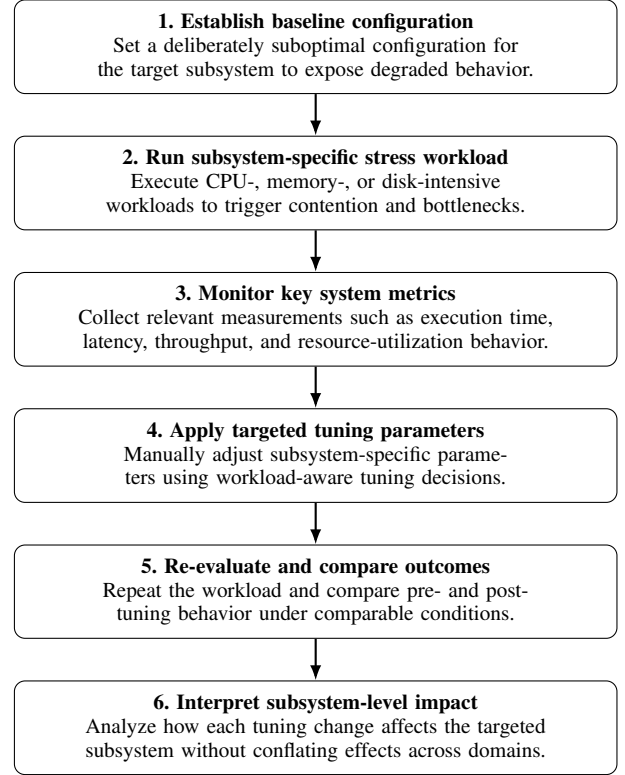


Fig. 1. Common experimental pipeline followed across CPU, memory, and disk tuning experiments.

A. CPU

The CPU experiment studies core contention and scheduler preemption as two separate performance factors. To keep I/O effects and incidental runtime variation low, we use a deterministic CPU-bound workload. Controlled oversubscription is then created by running multiple concurrent processes on a limited set of cores, which produces measurable contention along with higher scheduling activity and preemption frequency under a fixed execution pattern. The three CPU experiment configurations, denoted SB, C1, and C2, are listed in Table II.

TABLE II
CPU EXPERIMENT CONFIGURATIONS

Config.	Description
SB	Stress baseline; all concurrent processes pinned to a single core
C1	Affinity tuning; the same processes distributed across two cores
C2	Affinity and scheduler tuning; C1 placement with real-time scheduling applied to the foreground process

TABLE III
CPU EXPERIMENTAL DESIGN PARAMETERS

Parameter	Description
Workload type	Deterministic CPU-bound workload with negligible I/O activity
Contention model	Controlled oversubscription using multiple concurrent processes on limited cores
Workload consistency	Fixed workload size and identical execution structure across runs
Validation	Repeated runs with significance testing, including Welch's <i>t</i> -test where unequal variance assumptions are appropriate

The design separates the effects of affinity and scheduling. Comparing SB with C1 shows the impact of hardware-level core placement, while comparing C1 with C2 mainly captures OS-level scheduling and preemption behavior under the same affinity layout. Scheduler tuning may also affect effective CPU time allocation, not only preemption behavior.

To improve reliability, contention levels were kept controlled, the workload remained identical across runs, and experiments were repeated for statistical evaluation. Each CPU configuration is run 10 times; these repeated runs reduce the effect of transient noise, and the averaged values give a more stable estimate of steady-state behavior. Statistical validation is performed using multiple runs and appropriate significance testing methods, including Welch's *t*-test [10] when unequal variance assumptions are warranted.

B. Memory

The memory experiment looks at how changing kernel memory settings affects two types of tasks that use a lot of memory, when the system is under constant memory pressure.

The first task type, called the "alloc" workload, keeps allocating memory beyond what the system's RAM can hold. This mimics real-world applications like Java programs and software that heavily uses malloc.

The second task type, called the "cache" workload, reads memory in a sequence. This mimics databases and file servers where the main concern is how the system handles cache eviction.

Both tasks are run using a tool called stress-ng [11] for 90 seconds with 4 jobs running at the same time.

The baseline setup uses settings that are intentionally bad for memory-heavy tasks, as shown in Table IV. These settings make the kernel remove anonymous pages from memory too soon even when RAM is still free, clear the page cache earlier than needed, and slow down foreground write operations by flushing dirty pages too early.

Tuning covers five areas: swappiness, dirty write-back thresholds, vfs cache pressure, minimum free memory, and Transparent Huge Pages. Swappiness is set to 10, dirty_ratio to 20, dirty_background_ratio to 10, and min_free_kbytes to 262144 for all workloads as a strong base tuning. The remaining two parameters change depending on the workload type: for swap-heavy workloads, vfs_cache_pressure is set to

TABLE IV
BASELINE MEMORY PARAMETER VALUES

Parameter	Value	Effect
vm.swappiness	200	Aggressively evicts anonymous pages even when RAM is available
vm.vfs_cache_pressure	500	Forces premature page cache eviction
vm.dirty_ratio	5	Flushes dirty pages too eagerly, causing write stalls
vm.dirty_background_ratio	2	Background flusher too aggressive, interrupting foreground writes
vm.min_free_kbytes	16384	Too low, delaying background reclaim and causing sudden stalls
THP	never	Disables huge pages, increasing TLB pressure for large allocations

100 and Transparent Huge Pages are turned on to reduce TLB pressure on large anonymous memory allocations; for fault-heavy workloads, vfs_cache_pressure is lowered to 50 and Transparent Huge Pages are set to madvise to avoid compaction overhead. The workload type is decided based on swap rates and major page fault frequency seen during the baseline run.

The metrics collected during each run are listed in Table V. They come from three sources: free RAM and cached memory from /proc/meminfo, major page fault rate and swap rates from /proc/vmstat and vmstat, and memory stall time from the PSI interface at /proc/pressure/memory. A combined memory pressure score is calculated as a weighted average that focuses on the most important performance bottlenecks: CPU iowait (50 percent), major page fault rate (35 percent), swap-in rate (10 percent), and PSI memory stall time (5 percent). Swap-out rate is left out of the combined score on purpose to avoid unfairly penalizing the bursty, delayed eviction behavior that comes with a low-swappiness setup.

TABLE V
METRICS COLLECTED DURING MEMORY EXPERIMENTS

Metric	Source	What it measures
avg_free_mb	/proc/meminfo	Average free RAM; lower values indicate higher pressure
avg_pgmaifault	/proc/vmstat	Major page faults per second; page fetched from disk
avg_si_kBps	vmstat si	Pages swapped in per second in KB
avg_so_kBps	vmstat so	Pages swapped out per second in KB
avg_cache_mb	/proc/meminfo	Cached and buffered memory in MB; lower values indicate cache eviction
psi_some_avg10	/proc/pressure/memory	Percentage of time at least one task was stalled on memory over a 10-second window
memory_pressure_score	Derived	Weighted composite of iowait, major fault rate, swap-in rate, and PSI stall time

C. Disk

The disk methodology follows a controlled and repeatable process for analyzing and tuning Linux disk I/O performance. Each experiment starts with an intentionally non-optimized baseline configuration. A workload is then executed using fio [12] while system behavior is monitored. Workload characteristics are extracted, rule-based tuning is applied, and the same workload is executed again to compare results. Direct I/O is used in all experiments to minimize page-cache effects, and caches are flushed between runs to maintain consistency across baseline and tuned measurements.

TABLE VI
DISK TUNING PIPELINE AND INSTRUMENTATION

Stage	Component	Description
1	Baseline setup	Apply intentionally suboptimal scheduler and read-ahead values
2	Workload execution	Run fio workloads (rand/seq/mix) under controlled parameters
3	Monitoring	Collect latency, throughput, queue depth, utilization, iowait, PSI, request-size, and write-ratio data using iostat, vmstat, pidstat, and PSI
4	Feature extraction	Aggregate logs into structured workload and performance features
5	Classification	Determine workload type and disk-pressure characteristics
6	Tuning recommendation	Select scheduler and read-ahead using decision rules
7	Validation	Re-run workload under tuned configuration and compare outcomes

Workload type is inferred from observed request size and read/write ratio. A workload is treated as mixed when $\text{write_ratio} > 0.15$, sequential when $\text{avg_req_size} > 128$ KB, and random when $\text{avg_req_size} < 32$ KB. Scheduler selection is queue-aware: mq-deadline is used when $\text{avg_queue} > 50$, otherwise none is retained. Read-ahead is then assigned according to workload class: 1024 for sequential, 512 for mixed, and 128 for random access.

The workload-classification dimensions used in this study, such as request size, read/write ratio, and queue depth, follow commonly used disk-performance characterization practices described in Linux performance-analysis literature [16]. The threshold values were chosen empirically from preliminary test runs to create clear and practical workload categories for controlled evaluation.

Disk measurements were collected for each workload. Direct I/O and cache flushing were used to minimize variability caused by page caching and warm-state effects. The observed improvements, particularly in tail latency and PSI pressure metrics, provide a reliable indication of the impact of the applied tuning strategies.

III. RESULTS AND INSIGHTS

This section presents the observed performance behavior of the CPU, memory, and disk subsystems under the tuning

TABLE VII
DISK TUNING STRATEGY

Workload Type	Scheduler	Read-Ahead	Design Principle
Random	mq-deadline	128	Apply scheduler and read-ahead tuning to reduce wasteful prefetch and improve locality
Sequential	mq-deadline	1024	Apply scheduler and read-ahead tuning to maximize prefetch efficiency and throughput
Mixed	mq-deadline	512	Apply scheduler and read-ahead tuning to balance read/write contention and prefetch

strategies described earlier.

A. CPU

Table VIII summarizes the execution time across all three CPU configurations. In SB, behavior is dominated by deterministic single-core time-sharing: six CPU-bound processes compete for one core, so the foreground process spends much of its wall time waiting for CPU service. The fixed workload and regular scheduler cycles also explain the extremely low variance observed in SB.

TABLE VIII
CPU EXECUTION-TIME RESULTS

Cfg.	Mean (s)	SD (s)	CV	Change vs SB
SB	43.720	0.018	0.04%	—
C1	22.288	0.992	4.45%	−49.0%
C2	8.455	0.838	9.91%	−80.7%

C1 improves execution time because affinity changes how work is allocated at the hardware level. The same runnable set is spread across two cores, which gives the foreground process a larger effective CPU share. The scheduler itself, however, is unchanged.

TABLE IX
CPU CONTEXT-SWITCH RESULTS

Cfg.	Mean	SD	Change vs SB
SB	2470	5	—
C1	2449	19	−0.9%
C2	24	11	−99.0%

This is why C1 leaves context-switch behavior almost unchanged: reducing contention does not by itself change the preemption policy. Since multiple SCHED_OTHER tasks still

share each core, the foreground process continues to rotate under normal fair-share scheduling.

C2 changes the operating regime because `SCHED_RR` directly modifies OS-level scheduling. The foreground process no longer follows the same fair-share rotation; priority-based preemption lets it run for longer uninterrupted intervals, sharply reducing context switches and improving CPU-capture efficiency. The additional reduction in execution time therefore comes from fewer involuntary interruptions and more effective access to CPU service.

The CPU results show the control hierarchy clearly: affinity is a hardware-level knob, scheduling policy is an OS-level knob, and the two are orthogonal. Affinity improves processor allocation, but it does not reduce preemption overhead. Scheduler tuning becomes meaningful only when contention is high enough for preemption costs to be visible.

TABLE X
STATISTICAL VALIDATION FOR CPU RESULTS

Comparison	Metric	t	p -value	Cohen's d
SB vs C1	Wall time	68.34	< 0.001	30.56
C1 vs C2	Wall time	33.69	< 0.001	15.07
SB vs C1	Ctx switches	3.49	0.006	1.56
C1 vs C2	Ctx switches	350.22	< 0.001	156.62

Welch's t -test is appropriate here because the compared conditions do not share a common variance structure. SB is nearly deterministic, while C1 and C2 show larger spread due to cold-start and interrupt effects. The large t statistics and Cohen's d values occur because the differences between means are large relative to the within-condition variance, which is common in tightly controlled systems experiments with strong effects.

B. Memory

The memory experiment tested two workloads — alloc and cache — each run for 5 iterations of 90 seconds and averaged to reduce variability in Linux memory management and swap I/O behavior. Both workloads allocated 10 GB across 4 workers on an 8 GB RAM system, which forced real memory oversubscription and kept swap activity going throughout.

1) *Alloc Workload*: The alloc workload tested anonymous memory allocation using the walk-1d sequential access pattern. The baseline setup forced aggressive page eviction, which led to around 179,000 page faults per second and constant swap thrashing. After tuning, page faults dropped by 92.7 percent, CPU iowait went down by 12.4 percent, and throughput went up by 16.7 percent. The composite memory pressure score dropped by 49.9 percent. This improvement came mainly from keeping frequently used anonymous pages in RAM by lowering swappiness to 10 and raising `min_free_kbytes` to 262144, which caused smooth background memory reclaim to start early instead of sudden stalls happening late.

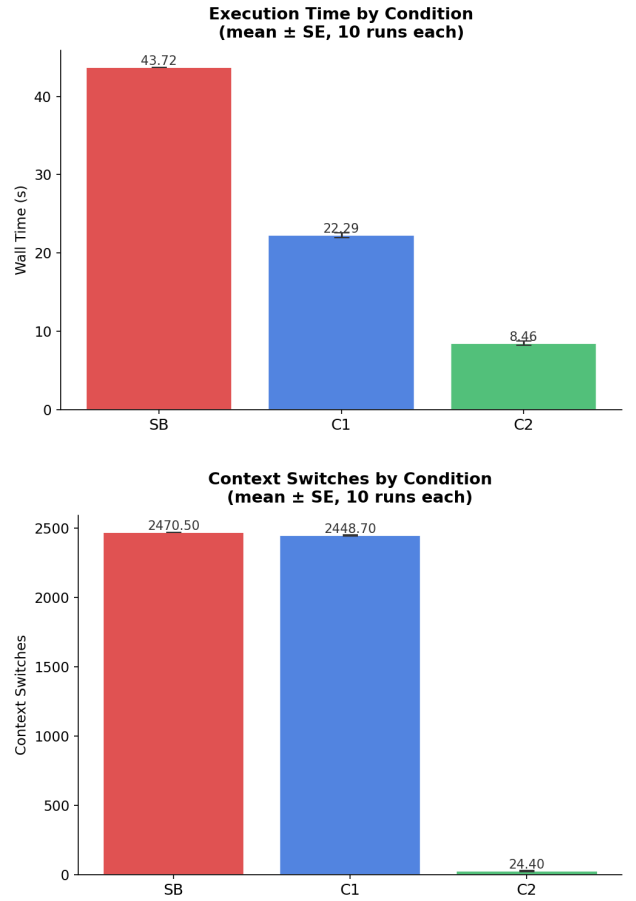


Fig. 2. CPU execution-time and context-switch behavior across configurations. Top: execution time. Bottom: foreground-process context switches.

2) *Cache Workload*: The cache workload also used the walk-1d sequential access pattern to match the kernel's LRU prediction logic. The baseline setup combined high `vfs_cache_pressure` with high swappiness, which made the kernel keep switching between disk cache and swap. After tuning, page faults dropped by 92.1 percent, CPU iowait went down by 38.9 percent, and throughput went up by 24.2 percent. The composite memory pressure score dropped by 73.7 percent. Keeping the page cache intact by lowering `vfs_cache_pressure` to 50 stopped repeated cache eviction and let sequential access patterns take full advantage of cache residency.

3) *Summary*: Table XI summarizes the averaged results across both workloads. Both configurations showed consistent and significant improvement across all four metrics after tuning, confirming that targeted kernel memory parameter changes produce measurable performance gains under memory oversubscription.

C. Disk

The disk results confirm that workload-aware tuning improves throughput, reduces latency, and lowers I/O pressure across random, sequential, and mixed workloads. Tables XII–

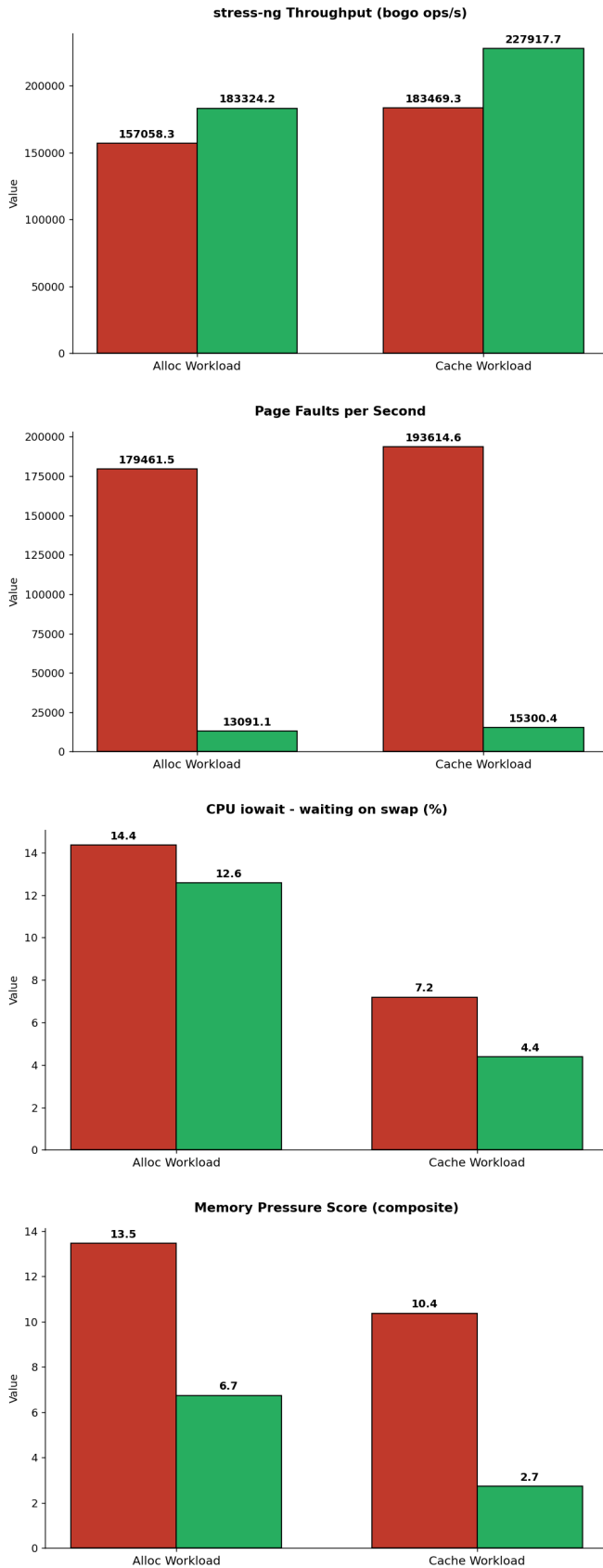


Fig. 3. Memory experiment results: throughput, page faults, CPU iowait, and composite pressure score averaged across 5 iterations for alloc and cache workloads. Green bars denote after tuning, and red bars denote before tuning.

TABLE XI
MEMORY EXPERIMENT RESULTS SUMMARY

Workload	Throughput	Page Faults	CPU iowait	Pressure Score
Alloc	+16.7%	−92.7%	−12.4%	−49.9%
Cache	+24.2%	−92.1%	−38.9%	−73.7%

XIV summarize the most representative before-versus-after metrics for each workload class.

TABLE XII
DISK RESULTS FOR THE RANDOM WORKLOAD

Metric	Baseline	Tuned	Change
IOPS	2047	2801	+36.8%
Bandwidth	8189 KiB/s	11161 KiB/s	+36.3%
Avg. latency	121.7 ms	90.8 ms	−25.3%
P99.9 latency	3071 ms	481 ms	−84.3%
PSI I/O pressure	62.81	20.63	−67.2%

For the random workload, tuning helped because the baseline combined the `none` scheduler with excessively high read-ahead, which wastes prefetch bandwidth for 4K random reads. Using `mq-deadline` with low read-ahead improved request ordering and sharply reduced tail latency. No important tradeoff was observed; the gains are broad and consistent.

TABLE XIII
DISK RESULTS FOR THE SEQUENTIAL WORKLOAD

Metric	Baseline	Tuned	Change
IOPS	424	707	+66.7%
Bandwidth	424 MiB/s	707 MiB/s	+66.7%
Avg. latency	296.9 ms	180.8 ms	−39.1%
P99.9 latency	8356 ms	609 ms	−92.7%
PSI I/O pressure	60.78	22.37	−63.2%

For the sequential workload, tuning helped because the baseline starved the prefetch pipeline with read-ahead 32. Raising read-ahead to 1024 and using `mq-deadline` greatly improved sustained throughput and removed severe tail stalls. A small tradeoff was observed in the full results: median latency increased slightly even though average latency and tail latency improved substantially.

For the mixed workload, tuning helped by using `mq-deadline` together with an intermediate read-ahead value that better balances read locality and write interference. This reduced contention for both access types and substantially improved tail latency. The tradeoff is that the mixed setting is a compromise; it does not optimize pure-read or pure-write behavior as aggressively as a single-purpose configuration would.

The figures above provide a visual comparison of tuning effects on the five disk-performance metrics discussed above.

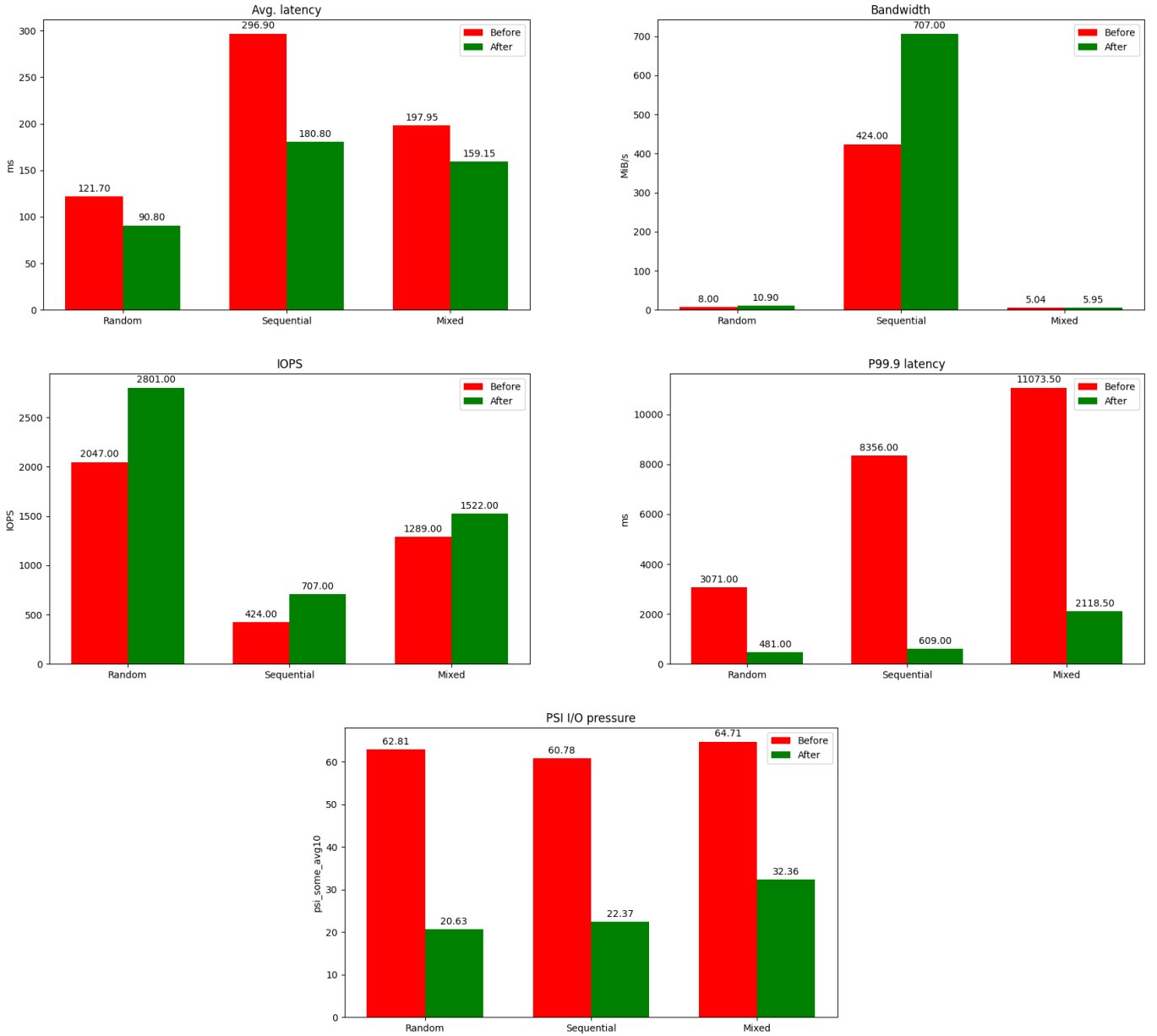


Fig. 4. Disk-performance comparison across random, sequential, and mixed workloads. The plots show average latency, bandwidth, IOPS, P99.9 latency, and PSI I/O pressure before and after tuning.

TABLE XIV
DISK RESULTS FOR THE MIXED WORKLOAD

Metric	Baseline	Tuned	Change
Read IOPS	900	1062	+18.0%
Write IOPS	389	460	+18.3%
Read P99.9 latency	10805 ms	1804 ms	−83.3%
Write P99.9 latency	11342 ms	2433 ms	−78.5%
PSI I/O pressure	64.71	32.36	−50.0%

IV. CONCLUSION

This study evaluated CPU, memory, and disk bottlenecks in Linux and showed that workload-aware tuning is more effective than generic configuration changes. Across CPU-, memory-, and I/O-intensive workloads, targeted tuning improved efficiency, reduced contention, and lowered system pressure. These results provide practical guidance for selecting simple, interpretable, and reproducible tuning strategies based on the dominant subsystem bottleneck.

Future work can build on this study in several practical directions. One possible extension is to move beyond manual tuning and develop a lightweight adaptive system that

monitors runtime metrics such as CPU contention, memory pressure, and disk queue activity, and applies tuning changes automatically as workloads change. Another direction is to study combined CPU, memory, and I/O pressure under mixed workloads instead of evaluating each subsystem separately, since tuning one subsystem may indirectly affect the others. It would also be useful to validate the proposed tuning strategies using production-oriented benchmark suites such as TPC-H [13], YCSB [14], and Filebench [15] to determine whether the observed improvements remain consistent outside controlled stress-testing environments.

ACKNOWLEDGMENT

The authors acknowledge the use of AI-assisted tools for limited code completion and language polishing. All experimental designs, measurements, analysis, and scientific conclusions are entirely the work of the authors.

REFERENCES

- [1] G. Whittaker, "Linux System Performance Tuning: Optimizing CPU, Memory, and Disk," *Linux Journal*, Mar. 4, 2025. [Online]. Available: <https://www.linuxjournal.com/content/linux-system-performance-tuning-optimizing-cpu-memory-and-disk>
- [2] D. Hoch, "Linux system and performance monitoring," *Penerbit O'Reilly*, USA, 2009.
- [3] A. J. Kumar, S. Korattiparambil, and P. B. Thangaraju, "Design and Implementation of an Advanced Continuous Monitoring System for Real-Time Performance Analytics," in *2025 IEEE International Conference on Recent Advances in Computing and Systems (REACS)*, Gwalior, India, 2025, pp. 1–8, doi: 10.1109/REACS67479.2025.11413605. Keywords: DevOps, data visualization, containers, real-time systems, CPU, predictive analytics, monitoring, optimization, thermal stability, system analysis and design, continuous monitoring, system metrics, Docker container monitoring, SAR, InfluxDB, Grafana, and real-time visualization.
- [4] M. D. Elradi, "Prometheus & Grafana: A Metrics-focused Monitoring Stack," *Journal of Computer Allied Intelligence*, 2025, n. pag.
- [5] L. Yi, "KernTune: Self-tuning Linux kernel performance using support vector machines," 2006.
- [6] Y. Sun et al., "KconfigTune: Automatic performance tuning for Linux kernel configuration," *IEEE Transactions on Computers*, 2025.
- [7] H. Lin et al., "OS-R1: Agentic operating system kernel tuning with reinforcement learning," arXiv preprint arXiv:2508.12551, 2025.
- [8] P. Cui, Z. Liu, and J. Bai, "Linux storage I/O performance optimization based on machine learning," in *2022 4th International Conference on Natural Language Processing (ICNLP)*, IEEE, 2022.
- [9] The Linux Kernel Organization, "Transparent Hugepage Support," *Linux Kernel Documentation*. [Online]. Available: <https://docs.kernel.org/admin-guide/mm/transhuge.html>
- [10] B. L. Welch, "The generalization of 'Student's' problem when several different population variances are involved," *Biometrika*, vol. 34, no. 1–2, pp. 28–35, 1947.
- [11] C. I. King, "stress-ng," GitHub repository. [Online]. Available: <https://github.com/ColinIanKing/stress-ng>.
- [12] J. Axboe, "fio: Flexible I/O Tester," [Online]. Available: <https://github.com/axboe/fio>
- [13] Transaction Processing Performance Council, "TPC Benchmark H (Decision Support) Standard Specification," TPC, 2014. [Online]. Available: <https://www.tpc.org/tpch>
- [14] Cooper, Brian F., et al. "Benchmarking cloud serving systems with YCSB." *Proceedings of the 1st ACM symposium on Cloud computing*. 2010.
- [15] Tarasov, Vasily, Erez Zadok, and Spencer Shepler. "Filebench: A flexible framework for file system benchmarking." *USENIX; login 41.1* (2016): 6–12.
- [16] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed., Pearson, 2020.